

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

Practical – VII

Roll No.: T016	Name: Himani Malankar
Class:TYIT	Batch:1
Date of Assignment: 01/08/2026	Date/Time of Submission: 01/08/2026

AIM: MACHINE LEARNING.

- 1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.**
 - a. Load dataset using Pandas.
 - b. Perform basic preprocessing (handling missing values or scaling).
 - c. Split the dataset into training and testing sets.
 - d. Train a simple classifier.
 - e. Display training and testing accuracy.

CODE A:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler
df = pd.read_csv("IRIS DATASET.csv")
print("\n===== 1. LOAD THE DATASET =====")
print(df)
print("\nHimani Malankar T016")
print("\n===== 2. DISPLAY THE DATASET =====")
print(df)
print("\nHimani Malankar T016")
print("\n===== 3. CHECK DATASET INFORMATION =====")
df.info()
print("\nHimani Malankar T016")
print("\n===== 4. DISPLAY DATASET SHAPE =====")
print(df.shape)
print("\nHimani Malankar T016")
print("\n===== 5. DISPLAY COLUMN NAMES =====")
print(df.columns)
print("\nHimani Malankar T016")
print("\n===== 6. DISPLAY SUMMARY STATISTICS =====")
print(df.describe())
print("\nHimani Malankar T016")
print("\n===== 7. CHECK DATA TYPES =====")
print(df.dtypes)
print("\nHimani Malankar T016")
print("\n===== 8. CHECK MISSING VALUES =====")
print(df.isnull().sum())
print("\nHimani Malankar T016")
print("\n===== 9. HANDLE MISSING VALUES =====")
df.fillna(df.mean(numeric_only=True), inplace=True)
print(df.isnull().sum())
```

```
print("\nHimani Malankar T016")
print("\n===== 10. REMOVE DUPLICATE RECORDS =====")
df.drop_duplicates(inplace=True)
print(df)
print("\nHimani Malankar T016")
print("\n===== 11. ENCODE CATEGORICAL DATA =====")
encoder = LabelEncoder()
df["species"] = encoder.fit_transform(df["species"])
print(df)
print("\nHimani Malankar T016")
print("\n===== 12. RENAME COLUMNS =====")
df.rename(columns={
    "sepal_length": "Sepal_Length",
    "sepal_width": "Sepal_Width",
    "petal_length": "Petal_Length",
    "petal_width": "Petal_Width",
    "species": "Species"
}, inplace=True)
print(df)
print("\nHimani Malankar T016")
print("\n===== 13. DROP UNNECESSARY COLUMNS =====")
df.drop(columns=[], inplace=True)
print(df)
print("\nHimani Malankar T016")
print("\n===== 14. SELECT FEATURES =====")
X = df.drop("Species", axis=1)
print(X)
print("\nHimani Malankar T016")
print("\n===== 15. CREATE NEW FEATURE =====")
df["Total_Length"] = df["Sepal_Length"] + df["Petal_Length"]
print(df)
print("\nHimani Malankar T016")
print("\n===== 16. CONVERT DATA TYPES =====")
df["Species"] = df["Species"].astype(int)
print(df.dtypes)
print("\nHimani Malankar T016")
print("\n===== 17. NORMALIZE DATA (MIN-MAX SCALING) =====")
scaler = MinMaxScaler()
normalized_data = scaler.fit_transform(X)
print(normalized_data)
print("\nHimani Malankar T016")
print("\n===== 18. STANDARDIZE DATA (STANDARD SCALING) =====")
standard_scaler = StandardScaler()
standardized_data = standard_scaler.fit_transform(X)
print(standardized_data)
print("\nHimani Malankar T016")
print("\n===== 19. SPLIT FEATURES AND TARGET VARIABLE =====")
X = df.drop(["Species"], axis=1)
y = df["Species"]
print("Features (X):")
print(X)
print("\nTarget (y):")
print(y)
print("\nHimani Malankar T016")
```

OUTPUT:

===== 1. LOAD THE DATASET =====

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

===== 2. DISPLAY THE DATASET =====

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

[150 rows x 5 columns]

Himani Malankar T016

===== 3. CHECK DATASET INFORMATION =====

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 150 entries, 0 to 149

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
---	----	-----	----

0	sepal_length	150 non-null	float64
1	sepal_width	150 non-null	float64
2	petal_length	150 non-null	float64
3	petal_width	150 non-null	float64
4	species	150 non-null	object

dtypes: float64(4), object(1)

memory usage: 6.0+ KB

Himani Malankar T016

===== 4. DISPLAY DATASET SHAPE =====

(150, 5)

Himani Malankar T016

===== 5. DISPLAY COLUMN NAMES =====

```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
      'species'],
      dtype='object')
```

Himani Malankar T016

===== 6. DISPLAY SUMMARY STATISTICS =====

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Himani Malankar T016

===== 7. CHECK DATA TYPES =====

```
sepal_length    float64
sepal_width     float64
petal_length     float64
petal_width     float64
species         object
dtype: object
```

Himani Malankar T016

===== 8. CHECK MISSING VALUES =====

```
sepal_length  0
sepal_width   0
petal_length  0
petal_width   0
species       0
dtype: int64
```

Himani Malankar T016

===== 9. HANDLE MISSING VALUES =====

```
sepal_length  0
sepal_width   0
petal_length  0
petal_width   0
species       0
dtype: int64
```

Himani Malankar T016

===== 10. REMOVE DUPLICATE RECORDS =====

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

[147 rows x 5 columns]

Himani Malankar T016

===== 11. ENCODE CATEGORICAL DATA =====

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

[147 rows x 5 columns]

Himani Malankar T016

===== 12. RENAME COLUMNS =====

	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

[147 rows x 5 columns]

Himani Malankar T016

===== 13. DROP UNNECESSARY COLUMNS =====

	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

[147 rows x 5 columns]

Himani Malankar T016

```
===== 14. SELECT FEATURES =====
Sepal_Length Sepal_Width Petal_Length Petal_Width
0      5.1      3.5      1.4      0.2
1      4.9      3.0      1.4      0.2
2      4.7      3.2      1.3      0.2
3      4.6      3.1      1.5      0.2
4      5.0      3.6      1.4      0.2
..      ...      ...      ...      ...
145     6.7      3.0      5.2      2.3
146     6.3      2.5      5.0      1.9
147     6.5      3.0      5.2      2.0
148     6.2      3.4      5.4      2.3
149     5.9      3.0      5.1      1.8

[147 rows x 4 columns]

Himani Malankar T016
```

```
===== 15. CREATE NEW FEATURE =====
Sepal_Length Sepal_Width Petal_Length Petal_Width Species \
0      5.1      3.5      1.4      0.2      0
1      4.9      3.0      1.4      0.2      0
2      4.7      3.2      1.3      0.2      0
3      4.6      3.1      1.5      0.2      0
4      5.0      3.6      1.4      0.2      0
..      ...      ...      ...      ...      ...
145     6.7      3.0      5.2      2.3      2
146     6.3      2.5      5.0      1.9      2
147     6.5      3.0      5.2      2.0      2
148     6.2      3.4      5.4      2.3      2
149     5.9      3.0      5.1      1.8      2

Total_Length
```

```
===== 16. CONVERT DATA TYPES =====
Sepal_Length  float64
Sepal_Width   float64
Petal_Length  float64
Petal_Width   float64
Species       int64
Total_Length  float64
dtype: object

Himani Malankar T016
```

```
===== 17. NORMALIZE DATA (MIN-MAX SCALING) =====
[[0.22222222 0.625      0.06779661 0.04166667]
 [0.16666667 0.41666667 0.06779661 0.04166667]
 [0.11111111 0.5       0.05084746 0.04166667]
 [0.08333333 0.45833333 0.08474576 0.04166667]
 [0.19444444 0.66666667 0.06779661 0.04166667]
 [0.30555556 0.79166667 0.11864407 0.125     ]
 [0.08333333 0.58333333 0.06779661 0.08333333]
 [0.19444444 0.58333333 0.08474576 0.04166667]
```

```
===== 18. STANDARDIZE DATA (STANDARD SCALING) =====
[[-0.91550949  1.0199714 -1.35773692 -1.33570001]
 [-1.15755966 -0.12808217 -1.35773692 -1.33570001]
 [-1.39960983  0.33113926 -1.41477817 -1.33570001]
 [-1.52063491  0.10152855 -1.30069567 -1.33570001]
 [-1.03653457  1.24958212 -1.35773692 -1.33570001]
 [-0.55243423  1.93841426 -1.18661318 -1.07090176]
 [-1.52063491  0.79036069 -1.35773692 -1.20330089]
 [-1.03653457  0.79036069 -1.30069567 -1.33570001]
```

===== 19. SPLIT FEATURES AND TARGET VARIABLE =====					
Features (X):					
	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width	Total_Length
0	5.1	3.5	1.4	0.2	6.5
1	4.9	3.0	1.4	0.2	6.3
2	4.7	3.2	1.3	0.2	6.0
3	4.6	3.1	1.5	0.2	6.1
4	5.0	3.6	1.4	0.2	6.4
..	...	...	...	...	...
145	6.7	3.0	5.2	2.3	11.9
146	6.3	2.5	5.0	1.9	11.3
147	6.5	3.0	5.2	2.0	11.7
148	6.2	3.4	5.4	2.3	11.6
149	5.9	3.0	5.1	1.8	11.0

Target (y):	
0	0
1	0
2	0
3	0
4	0
..	
145	2
146	2
147	2
148	2
149	2
Name: Species, Length: 147, dtype: int64	
Himani Malankar T016	

CODE B:

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler
df = pd.read_csv("supermarket_sales.csv")
print("\n===== 1. LOAD THE DATASET =====")
print(df)
print("\nHimani Malankar T016")
print("\n===== 2. DISPLAY THE DATASET =====")
print(df)
print("\nHimani Malankar T016")
print("\n===== 3. CHECK DATASET INFORMATION =====")
df.info()
print("\nHimani Malankar T016")
print("\n===== 4. DISPLAY DATASET SHAPE =====")
print(df.shape)
print("\nHimani Malankar T016")
print("\n===== 5. DISPLAY COLUMN NAMES =====")
print(df.columns)
print("\nHimani Malankar T016")
print("\n===== 6. DISPLAY SUMMARY STATISTICS =====")
print(df.describe())
print("\nHimani Malankar T016")
print("\n===== 7. CHECK DATA TYPES =====")
print(df.dtypes)
print("\nHimani Malankar T016")
print("\n===== 8. CHECK MISSING VALUES =====")
print(df.isnull().sum())
print("\nHimani Malankar T016")
print("\n===== 9. HANDLE MISSING VALUES =====")
df.fillna(df.select_dtypes(include="number").mean(), inplace=True)
print(df.isnull().sum())
print("\nHimani Malankar T016")
print("\n===== 10. REMOVE DUPLICATE RECORDS =====")
df.drop_duplicates(inplace=True)
print(df.shape)
print("\nHimani Malankar T016")
print("\n===== 11. ENCODE CATEGORICAL DATA =====")
encoder = LabelEncoder()
categorical_columns = [
    "Branch",
    "City",
    "Customer type",
    "Gender",
    "Product line",
    "Payment"
]
for col in categorical_columns:
    df[col] = encoder.fit_transform(df[col])
print(df.head())
print("\nHimani Malankar T016")
print("\n===== 12. RENAME COLUMNS =====")
df.rename(columns={
    "Invoice ID": "Invoice_ID",
    "Customer type": "Customer_Type",
    "Product line": "Product_Line",
    "Unit price": "Unit_Price",
    "Tax 5%": "Tax_5",
    "gross margin percentage": "Gross_Margin_Percentage",
    "gross income": "Gross_Income"
}, inplace=True)
print(df.columns)
print("\nHimani Malankar T016")
print("\n===== 13. DROP UNNECESSARY COLUMNS =====")
df.drop("Invoice_ID", axis=1, inplace=True)

```

```

print(df.head())
print("\nHimani Malankar T016")
print("\n===== 14. SELECT FEATURES =====")
X = df.drop("Rating", axis=1)
print(X.head())
print("\nHimani Malankar T016")
print("\n===== 15. CREATE NEW FEATURE =====")
df["Total_Amount"] = df["Unit_Price"] * df["Quantity"]
print(df.head())
print("\nHimani Malankar T016")
print("\n===== 16. CONVERT DATA TYPES =====")
df["Quantity"] = df["Quantity"].astype(float)
print(df.dtypes)
print("\nHimani Malankar T016")
print("\n===== 17. NORMALIZE DATA (MIN-MAX SCALING) =====")
numeric_columns = df.select_dtypes(include="number").columns
minmax = MinMaxScaler()
df[numeric_columns] = minmax.fit_transform(df[numeric_columns])
print(df.head())
print("\nHimani Malankar T016")
print("\n===== 18. STANDARDIZE DATA (STANDARD SCALING) =====")
standard = StandardScaler()
df[numeric_columns] = standard.fit_transform(df[numeric_columns])

print(df.head())
print("\nHimani Malankar T016")
print("\n===== 19. SPLIT FEATURES AND TARGET VARIABLE =====")
X = df.drop("Rating", axis=1)
y = df["Rating"]
print("Features (X)")
print(X.head())
print("\nTarget (y)")
print(y.head())
print("\nHimani Malankar T016")

```

OUTPUT:

===== 1. LOAD THE DATASET =====

DataSet Loaded.

Himani Malankar T016

===== 2. DISPLAY THE DATASET =====

	Invoice ID	Branch	City	Customer type	Gender \
0	750-67-8428	A	Yangon	Member	Female
1	226-31-3081	C	Naypyitaw	Normal	Female
2	631-41-3108	A	Yangon	Normal	Male
3	123-19-1176	A	Yangon	Member	Male
4	373-73-7910	A	Yangon	Normal	Male

	Product line	Unit price	Quantity	Tax 5%	Total	Date \
0	Health and beauty	74.69	7	26.1415	548.9715	1/5/2019
1	Electronic accessories	15.28	5	3.8200	80.2200	3/8/2019
2	Home and lifestyle	46.33	7	16.2155	340.5255	3/3/2019
3	Health and beauty	58.22	8	23.2880	489.0480	1/27/2019
4	Sports and travel	86.31	7	30.2085	634.3785	2/8/2019

	Time	Payment	cogs	gross margin percentage	gross income	Rating
0	13:08	Ewallet	522.83	4.761905	26.1415	9.1
1	10:29	Cash	76.40	4.761905	3.8200	9.6
2	13:23	Credit card	324.31	4.761905	16.2155	7.4
...						
4	-0.973790					

Name: Rating, dtype: float64

===== 3. CHECK DATASET INFORMATION =====

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Invoice ID             1000 non-null   object
1   Branch                 1000 non-null   object
2   City                   1000 non-null   object
3   Customer type          1000 non-null   object
4   Gender                 1000 non-null   object
5   Product line           1000 non-null   object
6   Unit price             1000 non-null   float64
7   Quantity               1000 non-null   int64
8   Tax 5%                 1000 non-null   float64
9   Total                  1000 non-null   float64
10  Date                   1000 non-null   object
11  Time                   1000 non-null   object
12  Payment                1000 non-null   object
13  cogs                   1000 non-null   float64
14  gross margin percentage 1000 non-null   float64
15  gross income            1000 non-null   float64
16  Rating                 1000 non-null   float64
dtypes: float64(7), int64(1), object(9)

```


===== 4. DISPLAY DATASET SHAPE =====

(1000, 17)

Himani Malankar T016

===== 5. DISPLAY COLUMN NAMES =====

```
Index(['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender',
      'Product line', 'Unit price', 'Quantity', 'Tax 5%', 'Total', 'Date',
      'Time', 'Payment', 'cogs', 'gross margin percentage', 'gross income',
      'Rating'],
      dtype='object')
```

Himani Malankar T016

===== 6. DISPLAY SUMMARY STATISTICS =====

	Unit price	Quantity	Tax 5%	Total	cogs \
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	55.672130	5.510000	15.379369	322.966749	307.58738
std	26.494628	2.923431	11.708825	245.885335	234.17651
min	10.080000	1.000000	0.508500	10.678500	10.17000
25%	32.875000	3.000000	5.924875	124.422375	118.49750
50%	55.230000	5.000000	12.088000	253.848000	241.76000
75%	77.935000	8.000000	22.445250	471.350250	448.90500
max	99.960000	10.000000	49.650000	1042.650000	993.00000

	gross margin percentage	gross income	Rating
count	1000.000000	1000.000000	1000.000000
mean	4.761905	15.379369	6.97270
std	0.000000	11.708825	1.71858
min	4.761905	0.508500	4.00000
25%	4.761905	5.924875	5.50000
50%	4.761905	12.088000	7.00000
75%	4.761905	22.445250	8.50000
max	4.761905	49.650000	10.00000

Himani Malankar T016

===== 7. CHECK DATA TYPES =====

Invoice ID	object
Branch	object
City	object
Customer type	object
Gender	object
Product line	object
Unit price	float64
Quantity	int64
Tax 5%	float64
Total	float64
Date	object
Time	object
Payment	object
cogs	float64
gross margin percentage	float64
gross income	float64
Rating	float64
dtype:	object

Himani Malankar T016

===== 8. CHECK MISSING VALUES =====

Invoice ID	0
Branch	0
City	0
Customer type	0
Gender	0
Product line	0
Unit price	0
Quantity	0
Tax 5%	0
Total	0
Date	0
Time	0
Payment	0
cogs	0
gross margin percentage	0
gross income	0
Rating	0
dtype:	int64

Himani Malankar T016

===== 9. HANDLE MISSING VALUES =====

Invoice ID	0
Branch	0
City	0
Customer type	0
Gender	0
Product line	0
Unit price	0
Quantity	0
Tax 5%	0
Total	0
Date	0
Time	0
Payment	0
cogs	0
gross margin percentage	0
gross income	0
Rating	0
dtype:	int64

Himani Malankar T016

===== 10. REMOVE DUPLICATE RECORDS =====

(1000, 17)

Himani Malankar T016

===== 11. ENCODE CATEGORICAL DATA =====

Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price \
0	750-67-8428	0	2	0	0	3 74.69
1	226-31-3081	2	1	1	0	0 15.28
2	631-41-3108	0	2	1	1	4 46.33
3	123-19-1176	0	2	0	1	3 58.22
4	373-73-7910	0	2	1	1	5 86.31

Quantity	Tax 5%	Total	Date	Time	Payment	cogs \
0	7 26.1415	548.9715	1/5/2019	13:08	2	522.83
1	5 3.8200	80.2200	3/8/2019	10:29	0	76.40
2	7 16.2155	340.5255	3/3/2019	13:23	1	324.31
3	8 23.2880	489.0480	1/27/2019	20:33	2	465.76
4	7 30.2085	634.3785	2/8/2019	10:37	2	604.17

	gross margin percentage	gross income	Rating
0	4.761905	26.1415	9.1
1	4.761905	3.8200	9.6
2	4.761905	16.2155	7.4
3	4.761905	23.2880	8.4
4	4.761905	30.2085	5.3

Himani Malankar T016

===== 12. RENAME COLUMNS =====

```
Index(['Invoice_ID', 'Branch', 'City', 'Customer_Type', 'Gender',
      'Product_Line', 'Unit_Price', 'Quantity', 'Tax_5', 'Total', 'Date',
      'Time', 'Payment', 'cogs', 'Gross_Margin_Percentage', 'Gross_Income',
      'Rating'],
      dtype='object')
```

Himani Malankar T016

===== 13. DROP UNNECESSARY COLUMNS =====

Branch	City	Customer_Type	Gender	Product_Line	Unit_Price	Quantity \
0	0	2	0	0	3	74.69 7
1	2	1	1	0	0	15.28 5
2	0	2	1	1	4	46.33 7
3	0	2	0	1	3	58.22 8
4	0	2	1	1	5	86.31 7

Tax_5	Total	Date	Time	Payment	cogs \
0	26.1415	548.9715	1/5/2019	13:08	2 522.83
1	3.8200	80.2200	3/8/2019	10:29	0 76.40
2	16.2155	340.5255	3/3/2019	13:23	1 324.31
3	23.2880	489.0480	1/27/2019	20:33	2 465.76
4	30.2085	634.3785	2/8/2019	10:37	2 604.17

Gross_Margin_Percentage	Gross_Income	Rating
0	4.761905	26.1415 9.1
1	4.761905	3.8200 9.6
2	4.761905	16.2155 7.4
3	4.761905	23.2880 8.4
4	4.761905	30.2085 5.3

Himani Malankar T016

===== 14. SELECT FEATURES =====

Branch	City	Customer_Type	Gender	Product_Line	Unit_Price	Quantity \
0	0	2	0	0	3	74.69 7
1	2	1	1	0	0	15.28 5
2	0	2	1	1	4	46.33 7
3	0	2	0	1	3	58.22 8
4	0	2	1	1	5	86.31 7

Tax_5	Total	Date	Time	Payment	cogs \
0	26.1415	548.9715	1/5/2019	13:08	2 522.83
1	3.8200	80.2200	3/8/2019	10:29	0 76.40
2	16.2155	340.5255	3/3/2019	13:23	1 324.31
3	23.2880	489.0480	1/27/2019	20:33	2 465.76
4	30.2085	634.3785	2/8/2019	10:37	2 604.17

Gross_Margin_Percentage	Gross_Income
0	4.761905 26.1415
1	4.761905 3.8200
2	4.761905 16.2155
3	4.761905 23.2880
4	4.761905 30.2085

Himani Malankar T016

===== 15. CREATE NEW FEATURE =====

Branch	City	Customer_Type	Gender	Product_Line	Unit_Price	Quantity \
0	0	2	0	0	3	74.69 7
1	2	1	1	0	0	15.28 5
2	0	2	1	1	4	46.33 7
3	0	2	0	1	3	58.22 8
4	0	2	1	1	5	86.31 7

Tax_5	Total	Date	Time	Payment	cogs \
0	26.1415	548.9715	1/5/2019	13:08	2 522.83
1	3.8200	80.2200	3/8/2019	10:29	0 76.40
2	16.2155	340.5255	3/3/2019	13:23	1 324.31
3	23.2880	489.0480	1/27/2019	20:33	2 465.76
4	30.2085	634.3785	2/8/2019	10:37	2 604.17

Gross_Margin_Percentage	Gross_Income	Rating	Total_Amount
0	4.761905	26.1415 9.1	522.83
1	4.761905	3.8200 9.6	76.40
2	4.761905	16.2155 7.4	324.31
3	4.761905	23.2880 8.4	465.76
4	4.761905	30.2085 5.3	604.17

Himani Malankar T016

===== 16. CONVERT DATA TYPES =====

```

Branch          int64
City            int64
Customer_Type   int64
Gender          int64
Product_Line    int64
Unit_Price      float64
Quantity        float64
Tax_5           float64
Total           float64
Date            object
Time            object
Payment         int64
cogs            float64
Gross_Margin_Percentage float64
Gross_Income    float64
Rating          float64
Total_Amount    float64
dtype: object

```

Himani Malankar T016

===== 17. NORMALIZE DATA (MIN-MAX SCALING) =====

```

Branch City Customer_Type Gender Product_Line Unit_Price Quantity \
0  0.0  1.0      0.0  0.0      0.6  0.718847  0.666667
1  1.0  0.5      1.0  0.0      0.0  0.057855  0.444444
2  0.0  1.0      1.0  1.0      0.8  0.403316  0.666667
3  0.0  1.0      0.0  1.0      0.6  0.535603  0.777778
4  0.0  1.0      1.0  1.0      1.0  0.848131  0.666667

Tax_5  Total  Date  Time  Payment  cogs \
0 0.521616 0.521616 1/5/2019 13:08 1.0 0.521616
1 0.067387 0.067387 3/8/2019 10:29 0.0 0.067387
2 0.319628 0.319628 3/3/2019 13:23 0.5 0.319628
3 0.463549 0.463549 1/27/2019 20:33 1.0 0.463549
4 0.604377 0.604377 2/8/2019 10:37 1.0 0.604377

Gross_Margin_Percentage Gross_Income Rating Total_Amount
0 0.0 0.521616 0.850000 0.521616
1 0.0 0.067387 0.933333 0.067387
2 0.0 0.319628 0.566667 0.319628
3 0.0 0.463549 0.733333 0.463549
4 0.0 0.604377 0.216667 0.604377

```

Himani Malankar T016

===== 18. STANDARDIZE DATA (STANDARD SCALING) =====

```

Branch City Customer_Type Gender Product_Line Unit_Price \
0 -1.208970 1.210174 -0.998002 -0.998002 0.319617 0.718160
1 1.238338 -0.009759 1.002002 -0.998002 -1.430109 -1.525303
2 -1.208970 1.210174 1.002002 1.002002 0.902859 -0.352781
3 -1.208970 1.210174 -0.998002 1.002002 0.319617 0.096214
4 -1.208970 1.210174 1.002002 1.002002 1.486101 1.156959

Quantity Tax_5 Total Date Time Payment cogs \
0 0.509930 0.919607 0.919607 1/5/2019 13:08 1.203528 0.919607
1 -0.174540 -0.987730 -0.987730 3/8/2019 10:29 -1.205937 -0.987730
2 0.509930 0.071446 0.071446 3/3/2019 13:23 -0.001205 0.071446
3 0.852165 0.675780 0.675780 1/27/2019 20:33 1.203528 0.675780
4 0.509930 1.267125 1.267125 2/8/2019 10:37 1.203528 1.267125

Gross_Margin_Percentage Gross_Income Rating Total_Amount
0 0.0 0.919607 1.238443 0.919607
1 0.0 -0.987730 1.529527 -0.987730
2 0.0 0.071446 0.248760 0.071446
3 0.0 0.675780 0.830927 0.675780
4 0.0 1.267125 -0.973790 1.267125

```

Himani Malankar T016

===== 19. SPLIT FEATURES AND TARGET VARIABLE =====

Features (X)

```

Branch City Customer_Type Gender Product_Line Unit_Price \
0 -1.208970 1.210174 -0.998002 -0.998002 0.319617 0.718160
1 1.238338 -0.009759 1.002002 -0.998002 -1.430109 -1.525303
2 -1.208970 1.210174 1.002002 1.002002 0.902859 -0.352781
3 -1.208970 1.210174 -0.998002 1.002002 0.319617 0.096214
4 -1.208970 1.210174 1.002002 1.002002 1.486101 1.156959

Quantity Tax_5 Total Date Time Payment cogs \
0 0.509930 0.919607 0.919607 1/5/2019 13:08 1.203528 0.919607
1 -0.174540 -0.987730 -0.987730 3/8/2019 10:29 -1.205937 -0.987730
2 0.509930 0.071446 0.071446 3/3/2019 13:23 -0.001205 0.071446
3 0.852165 0.675780 0.675780 1/27/2019 20:33 1.203528 0.675780
4 0.509930 1.267125 1.267125 2/8/2019 10:37 1.203528 1.267125

```

Feature	Iris Dataset	Supermarket Dataset
Dataset Type	Flower dataset	Sales transaction dataset
Number of Rows	150	1000
Number of Columns	5	17

Target Variable	Species	Rating (for practice)
Data Type	Mostly Numeric	Numeric + Categorical
Missing Values	Usually None	Usually None
Duplicate Records	Rare	May Exist
Encoding Needed	Species only	Branch, City, Gender, Payment, etc.
Scaling Needed	Yes	Yes
Feature Engineering	Total_Length	Total_Amount
Unnecessary Columns	None	Invoice_ID
Machine Learning Task	Classification	Classification / Regression
Complexity	Simple	Moderate
Best Used For	Learning ML Basics	Data Preprocessing & Sales Analysis